

Mechanistic Interpretability: Diagnosing and Mitigating Activation Quantization Damage via Sparse Autoencoders

Nidheesh Kannadasan

June 7, 2026

Abstract

Large language models encode rich semantic and contextual concepts within high-dimensional hidden representation spaces. While low-bit activation quantization dramatically reduces operational memory footprints and computational overhead, it often degrades downstream performance unpredictably. Rather than evaluating performance drops solely through macroscopic output degradation metrics like perplexity, this report presents a mechanistic investigation into how quantization alters the internal representation space of a Transformer. Using a frozen `distilgpt2` model, we train a Top- K Sparse Autoencoder (SAE) to extract interpretable, sparse features from internal hidden states. We systematically trace how uniform quantization distortion interacts with individual feature dimensions and geometry. We expose distinct structural vulnerabilities, including magnitude bias in linear approximations, sparsity fragility, and low-variance collapse. Leveraging these insights, we implement a novel Subspace-Preserving Quantization framework that isolates and protects critical geometric directions via singular value decomposition. This method secures a 25% reduction in the bottleneck memory footprint while preserving essential information geometry and stabilizing downstream perplexity.

1 Introduction & Theoretical Background

Large Language Models (LLMs) rely on high-dimensional hidden representations to handle complex contextual reasoning. Deploying these networks efficiently within constrained compute environments often requires low-bit quantization of activations and weights. Although quantization minimizes memory footprints and accelerates matrix operations, it introduces non-linear rounding errors that can severely degrade downstream model utility.

Traditional optimization workflows judge quantization schemes purely by monitoring macroscopic drops in language modeling perplexity. This top-down approach treats the transformer’s internal layers as a black box, offering little insight into *why* specific compression levels destroy representation structures.

This project adopts a mechanistic interpretability approach to dissect internal representation distortion. Instead of treating hidden states as dense, uninterpretable vectors, we use dictionary learning via Sparse Autoencoders (SAEs) to untangle the representation space into discrete, monosemantic features. This enables us to pinpoint exactly which latent subspaces are most vulnerable to quantization noise, understand how compression interacts with feature sparsity, and develop a targeted, geometry-aware mitigation strategy.

2 Computational Setup & Framework

2.1 Hardware Constraints

All experiments, training loops, and evaluation pipelines were executed locally on an NVIDIA RTX 4060 GPU with a strict hardware limit of 8GB VRAM running on an Ubuntu Linux environment.

2.2 Dataset Strategy

The assignment required several evaluation metrics with significantly different computational costs. As a result, separate datasets were allocated for training, calibration, perplexity evaluation, geometric analysis, and causal importance measurements. This allowed each experiment to be performed efficiently while remaining within the available hardware constraints:

1. **409.6M Top-K SAE Train Set:** Used for training the Top-K SAE. Note that in the assignment it was mentioned to use roughly 1% of the corpus (80-100M tokens), a batch size between 4k-8k and 100k steps. This created a three-way circular dependency which could not be met. In order to resolve this and considering 100k steps as a proxy for training duration, a batch size of 4096 was chosen due to computational constraints, leading to a train set of 409.6M tokens.
2. **204.8k tokens calibration set:** This dataset, a subset of the training set consisting of exactly 204,800 tokens, was used to compute min/max calibration statistics for Tasks 2 and 4.
3. **2M Tokens Held-out Stream:** Saved as `ppl_tokens_2M.pt`, this subset was used exclusively for computing downstream Perplexity where a large sample size is required for statistical convergence.
4. **10112 Tokens held out set:** Used in Task 2 to visualize geometry with UMAP.

5. **4096 Tokens Held-out Batch:** Saved as `sds_tokens.4096.pt`, this chunk was used for computing geometric and spectral metrics, including Singular Value Decomposition (SVD), Subspace Distortion Scores (SDS), Mean Squared Error (MSE), and Centered Kernel Alignment (CKA). Because the computational complexity of SVD scales as $\mathcal{O}(Nm^2)$, executing it on the full 2M token stream would instantly trigger an out-of-memory error.
6. **50k Tokens Held-out Subset:** Used for computing causal importance metrics (Fisher-style gradients). This subset is randomly sampled from a highly homogeneous partition of the OpenWebText corpus. The expected value of local gradients converges efficiently on this allocation, providing a reliable proxy for causal impact without requiring large-scale backward passes.

3 Task 1: Pipeline Setup & Sparse Autoencoder Training

3.1 Data Extraction & Preprocessing

Using the `TransformerLens` library, we initialized a frozen, pretrained `distilgpt2` model. We streamed tokens from a target 1% subset of the OpenWebText corpus, utilizing a 0.2% sub-split for initial pipeline debugging. Hidden states were intercepted and extracted from Layer 2 (0-indexed, corresponding to the third layer of the 6-layer architecture). To prevent structural edge artifacts, we restricted extraction to contiguous sequences of exactly 128 tokens, omitting all padded sequences and isolated-token samples. Prior to autoencoder training, the gathered hidden states (structured as `[batch, seq_len, 768]`) were normalized and completely shuffled along the batch and sequence dimensions to disrupt temporal correlations across adjacent tokens.

3.2 SAE Architecture & Optimization

We trained Top- K Sparse Autoencoders via the `SAELens` library to map the dense layer activations into an expanded, sparse feature space. The autoencoder restricts activation density by enforcing a hard 10% sparsity constraint ($k = 0.1m$). We evaluated two dictionary bottleneck capacities: $m \in \{512, 1024\}$, initializing training with $m = 512$. The forward pass, dictionary encoding, and reconstruction are governed by the following mathematical formulations:

$$z = E(x), \quad \hat{x} = D(z), \quad \mathcal{L}_{\text{SAE}} = \|x - \hat{x}\|_2^2 \quad (1)$$

$$\text{subject to } |\text{supp}(z)| \leq k, \quad \text{where } k = 0.1m \quad (2)$$

Optimization was executed using the Adam optimizer with a fixed learning rate $\eta = 10^{-4}$ and an operational batch size configured between 4k and 8k vectors. The optimization trajectory ran for exactly 100,000 steps.

3.3 Weights & Biases Monitoring Integration

All core training dynamics were streamed directly to Weights & Biases (W&B). Key tracked metrics included the structural Reconstruction Loss (MSE), Sparsity Loss profiles, the explicit stability of the L_0 norm, and dead neuron counts. For final comparative reporting, the dynamic W&B Reports feature was utilized to group, arrange, and evaluate training run

charts side-by-side with text cells, enabling interactive iteration prior to exporting figures and trends into this document.

4 Task 2: Quantization Analysis & Activation Distortion

4.1 Quantization Schemes and Calibration

We implemented uniform, linear quantization functions to evaluate the structural resilience of the learned dictionary spaces. The implementation supports both per-tensor and per-feature granularities across a staged sweep of bitwidths ($b \in \{8, 6, 4, 2\}$).

Critically, the quantization framework is restricted to an unsigned configuration spanning the integer range $[q_{\min}, q_{\max}] = [0, 2^b - 1]$. This unsigned constraint matches the underlying mathematics of the Top- K SAE architecture; because the dictionary encoder relies on a ReLU activation function combined with top- K sparsification, all valid bottleneck feature activations are strictly non-negative ($z \geq 0$). Designing a signed quantization grid would waste bits on empty negative regions.

Quantization scaling steps (Δ) are derived dynamically from min/max calibration statistics collected across a dedicated 204.8k token stream. To protect against division-by-zero anomalies in inactive feature channels, a numerical lower bound floor of 10^{-8} is enforced via clamping operations. The forward projection maps full-precision vectors to the integer domain and subsequently maps them back to reconstructed tensors according to:

$$\Delta_{\text{per_tensor}} = \max\left(\frac{z_{\text{global_max}}}{2^b - 1}, 10^{-8}\right) \quad (3)$$

$$\Delta_{\text{per_feature},j} = \max\left(\frac{z_{\text{feat_max},j}}{2^b - 1}, 10^{-8}\right) \quad (4)$$

$$\hat{z} = \text{clip}\left(\left\lfloor \frac{z}{\Delta} \right\rfloor, 0, 2^b - 1\right), \quad \tilde{z} = \Delta \hat{z} \quad (5)$$

4.2 Layer Intervention and Evaluation Architecture

Model evaluation intervenes directly on the Layer 2 residual stream hook location (`blocks.2.hook_resid_post`) using the model caching mechanics in `HookedTransformer`. Rather than quantizing raw dense layer states, the intervention isolates the dictionary bottleneck. The full-precision activation x is intercepted and mapped to the sparse representation $z = E(x)$. The chosen quantization scheme computes the lossy reconstructed tensor $\tilde{z}$, which is then passed to the decoder to form $\hat{x} = D(\tilde{z})$. This modified reconstruction replaces the original hidden state before the remaining forward pass executes.

Downstream performance degradation is tracked through global Perplexity using the 2M token stream (`pp1.tokens_2M.pt`). Simultaneously, local geometric and structural distortions are captured on the 4096 held-out token batch (`sds.tokens_4096.pt`) by computing the Mean Squared Error (MSE) of the representations and tracking the Subspace Distortion Score (SDS) relative to baseline unquantized singular vectors.

4.3 Implementation Insights & Technical Solutions

Computing the Linear Centered Kernel Alignment (CKA) across high-dimensional activation matrices introduced numerical instability and frequent NaN overflow errors during cross-matrix

dot products. To eliminate this behavior, the CKA script forces an explicit conversion of the feature tensors to `float64` double precision. Given centered and normalized activation matrices $\mathbf{X}_c$ and $\mathbf{Y}_c$, the implementation centers and computes the alignment score via:

$$\mathbf{X}_c = \mathbf{X} - \mu_{\mathbf{X}}, \quad \mathbf{Y}_c = \mathbf{Y} - \mu_{\mathbf{Y}} \quad (6)$$

$$\tilde{\mathbf{X}} = \frac{\mathbf{X}_c}{\|\mathbf{X}_c\|_F + 10^{-8}}, \quad \tilde{\mathbf{Y}} = \frac{\mathbf{Y}_c}{\|\mathbf{Y}_c\|_F + 10^{-8}} \quad (7)$$

$$\text{CKA}(\mathbf{X}, \mathbf{Y}) = \frac{\|\tilde{\mathbf{X}}^\top \tilde{\mathbf{Y}}\|_F^2}{\|\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\|_F \|\tilde{\mathbf{Y}}^\top \tilde{\mathbf{Y}}\|_F} \quad (8)$$

To track geometric variations, the activation maps are saved at specific checkpoints. The unquantized baseline states are preserved as `bottleneck_activations_m[size]_4096_fp.pt`. For spatial representation tracking via UMAP and *t*-SNE algorithms, the framework caches activation matrices spanning exactly 10,112 tokens (compiled from 79 individual 128-token context chunks), saved systematically under names following the convention `umap_activations_m[size]_10112_[met`

4.4 Results and Interpretation

Placeholder section for Plots and Analyzes

5 Task 3: Representation Damage and Causal Importance

5.1 Feature Distortion Metrics & Triplicated Ranking Engine

To map out how uniform quantization degradation splits across individual SAE dimensions, the processing pipeline evaluates feature distortion under a triplicated metric architecture. For each of the d_{sae} hidden units, structural distortion is tracked across the 2M token stream bit-width settings through two individual pathways: geometric coordinate displacement (L_2 distance) and probability distribution divergence (Kullback-Leibler divergence).

The feature-wise L_2 distortion score computes the root-mean-squared error between the original full-precision feature activation z_j and its corresponding quantized value $\tilde{z}_j$ across all processed tokens:

$$L_{2,j} = \sqrt{\frac{1}{N_{\text{tokens}}} \sum_{n=1}^{N_{\text{tokens}}} (z_{n,j} - \tilde{z}_{n,j})^2} \quad (9)$$

Simultaneously, the framework monitors shifts in activation probability distributions. For each neuron, activations are tracked using a discrete 100-bin histogram covering the range from 0 to the global calibration maximum. The full-precision activation density P is compared against the quantized activation density Q by calculating the discrete KL divergence. A numerical smoothing floor of $\epsilon = 10^{-8}$ is enforced to handle zero-probability bins without causing numerical errors:

$$D_{\text{KL}}(P_j \parallel Q_j) = \sum_{b=1}^{100} P_{j,b} \log \left(\frac{P_{j,b} + 10^{-8}}{Q_{j,b} + 10^{-8}} \right) \quad (10)$$

To identify the most heavily degraded features, the pipeline implements three separate ranking strategies:

1. **Pure L_2 Sorting:** Profiles dimensions based strictly on absolute geometric error magnitudes.
2. **Pure KL Sorting:** Prioritizes features experiencing the highest structural distribution distortion.
3. **Borda Count Aggregation:** Combines both perspectives by calculating a joint rank score. This is done by adding the index positions from the L_2 and KL sorting passes: $\mathcal{R}_{\text{Borda},j} = \text{Rank}_{L_2}(j) + \text{Rank}_{\text{KL}}(j)$, and sorting the resulting array in ascending order.

Because large language model representations contain many inactive or dead units, sorting operations encounter large blocks of identical 0.0 values. To handle these tied values reliably on the GPU, the codebase sets a global initialization seed via `set_seed(42)` and forces all index extractions to use stable sorting routines via `torch.argsort(..., stable=True)`.

5.2 Interpretive Feature Profiling & Contextual Window Slicing

To analyze what linguistic concepts are disrupted by low-bit quantization, the framework tracks and logs the top $k = 20$ activating token examples for the most heavily damaged hidden units. Rather than capturing only the individual active token, the extraction script saves a wide 9-token contextual field centered on the target token (capturing 4 context tokens to the left and 4 to the right). This approach mirrors the structural analysis methodologies in Anthropic’s interpretive literature to map operational boundaries.

An explicit zero-activation trap filter (`if vals[i] > 0.0`) drops empty text entries, ensuring that only true feature activations are indexed. To clearly show what triggered the feature within the saved token strings, the mid-point activation target is wrapped in explicit bracketing markers (`<< >>`).

Looking at the exported data logs, this context-window technique successfully maps semantic identities to individual hidden units. For instance, evaluating the JSON log outputs for feature 98 reveals a clear, specialized sensitivity to weekday subwords:

```
" popular vote. Earlier<< Saturday >>, he retweeted (act: 12.66)",
".\u201d Last<< Thursday >> night, as I (act: 12.64)",
" officials during a meeting<< Monday >>, has neither signed (act: 12.62)",
" Trump at a rally<< Monday >> evening at the First (act: 12.57)",
"ocque met last<< Thursday >> with a team from (act: 12.56)",
" says that this past<< Friday >> was the first time (act: 12.56)",
" towards legalization came last<< Monday >> in the form of (act: 12.53)",
" on home ice last<< Wednesday >>. They faced a (act: 12.46)",
" Trump meeting MORE said<< Monday >> that carbon dioxide emissions (act: 12.41)",
"-election campaign last<< Thursday >> for using a fire (act: 12.39)",
"<|endoftext|> the decision later<< Monday >>.\n\nHe (act: 12.38)",
" fully embraced Trump last<< Friday >>, didn't want (act: 12.36)",
"olution 112 passed last<< Monday >> proclaiming opposition to legalization (act: 12.36)",
"TX) revealed on<< Tuesday >> that he had wanted (act: 12.35)",
" the Berlin meetup<< yesterday >> and even though I (act: 12.30)",
" Lopez his job back<< yesterday >>, Lopez declined. (act: 12.28)",
" by a storm last<< Sunday >> from a cold front (act: 12.27)",
"<|endoftext|><|endoftext|><|endoftext|><|endoftext|><{yesterday} >>, the Los Angeles (act:"
```

"<|endoftext|><|endoftext|><|endoftext|><|endoftext|><{yesterday}>> against this amendment.
"<|endoftext|> the Yellowstone on<< Thursday >> evening when she walked (act: 12.15)"

This explicit structural tracking confirms that feature 98 functions as a monosemantic day-of-the-week detector.

5.3 Spectral and Subspace Distortion Analysis

To capture how lossy uniform compression affects the underlying representation geometry, the analysis script runs a formal singular value decomposition pass. The evaluation focuses on the centered 4096 held-out token activation matrix ($\mathbf{Z} \in \mathbb{R}^{4096 \times m}$), processed in full precision (`float32`) to guarantee numerical stability:

$$\mathbf{Z}_{\text{centered}} = \mathbf{Z} - \mu_{\mathbf{Z}}, \quad \mathbf{Z}_{\text{centered}} = \mathbf{U}_{\text{fp}} \mathbf{V}_{\text{fp}}^{\top} \quad (11)$$

The right singular vectors matrix $\mathbf{V}_{\text{fp}}$ is transposed and cached as $\mathbf{U}_k$ to serve as the geometric base reference for subsequent tasks.

For each of the 8 quantized configurations (spanning 4 bitwidths and 2 structural methods), the script centers the quantized activation matrix $\hat{\mathbf{Z}}$ and extracts its compressed right singular vectors $\mathbf{V}_q$ using SVD. Alignment changes between the full-precision and quantized spaces are tracked across rank thresholds ($k \in \{32, 64, 128\}$) by isolating the top- k column subspaces:

$$\mathbf{M} = \mathbf{U}_{k,\text{fp}}^{\top} \mathbf{U}_{k,q} \quad (12)$$

Rather than performing a full SVD on this intermediate alignment matrix $\mathbf{M}$, the implementation uses the faster `torch.linalg.svdvals(M)` to directly extract the singular values, $\sigma_i(\mathbf{M})$. To prevent floating-point drift from producing out-of-bounds inputs for downstream trigonometric steps, the singular values are clamped securely to the interval $[0.0, 1.0]$. The principal angles θ_i governing the rotation of the data geometry are computed as:

$$\theta_i = \arccos(\text{clamp}(\sigma_i(\mathbf{M}), 0.0, 1.0)) \quad (13)$$

To capture the overall structural deformation within these principal directions, the script calculates the Subspace Distortion Score (SDS). By hoisting the raw coordinate difference matrix ($\mathbf{Z}_{\text{diff}} = \mathbf{Z} - \hat{\mathbf{Z}}$) out of the inner loop to save memory, the score measures the ratio of lost variance to original variance within the targeted top- k full-precision directions:

$$\text{SDS}_k = \frac{\|\mathbf{Z}_{\text{diff}} \mathbf{U}_{k,\text{fp}}\|_F^2}{\|\mathbf{Z} \mathbf{U}_{k,\text{fp}}\|_F^2} \quad (14)$$

5.4 Causal Gradient Attribution vs. Ablation and Magnitude Bias

To bridge the gap between structural feature damage and actual downstream model degradation, the framework evaluates feature importance using a 3-way toggle architecture. While Toggles 1 and 2 rely on brute-force single-neuron zero-ablation loops, Toggle 3 implements a first-order Gradient Attribution protocol (a fast, Fisher-style scoring routine) run over a 50k token subset to stay within hardware limits.

The attribution pass tracks how individual feature dimensions drive the global language modeling loss $\mathcal{L}$. This gradient proxy isolates the absolute product of the feature activation

and its corresponding backward gradient:

$$\text{Attribution}_j = \frac{1}{N} \sum_n \left| z_{n,j} \cdot \frac{\partial \mathcal{L}}{\partial z_{n,j}} \right| \quad (15)$$

Our experiments revealed a clear **magnitude bias** inherent to local linear approximations. The gradient attribution scores showed an artificially inflated alignment with the L_2 distortion rankings, yielding a high Spearman rank correlation of ~ 0.77 .

This confirms the core limitations discussed in Anthropic’s *Scaling Monosemanticity* framework: first-order gradient attribution drops the non-linear term in Taylor expansions. Because attribution scales directly with raw activation amplitudes, it tends to misidentify high-magnitude features as causally vital, failing to capture the non-linear saturation effects that occur under heavy quantization noise.

5.5 Results and Interpretation

Placeholder Section: Insert task-wise comparative plots, CKA metrics, and detailed text analyses here.

6 Task 4, Part 1: Mechanistic Explanations (Why Quantization Fails)

6.1 Cross-Task Diagnostic Framework & Ingestion Pipeline

To reveal the mechanical root causes behind the destructive interaction between feature sparsity and uniform quantization grids, the diagnostic pipeline operates on a unified cross-task architecture. Implemented without active token streaming to optimize compute efficiency under strict VRAM footprints, the diagnostic script consolidates results from previous pipelines by directly ingesting serialized multi-task outputs from disk.

Perfect execution reproducibility is guaranteed by establishing a deterministic environment via `set_seed(42)`, tracking random states across PyTorch, NumPy, and CUDA backends. The analysis framework processes each dictionary capacity ($m \in \{512, 1024\}$) by coordinating three separate data files:

1. `attributions_m[size].pt`: Caches the explicit first-order Fisher gradient attribution vectors derived from the 50k token subset pass in Task 3.
2. `spectral_analysis_metrics.json`: Provides full-precision and quantized singular value spectra alongside projection profiles collected on the 4096-token batch.
3. `task3_metrics_method2.json`: Delivers feature-wise L_2 coordinate displacement and discrete D_{KL} distribution divergence arrays across all evaluated compression configurations.

6.2 Encoder Jacobian Sensitivity & Topological Fragility Mapping

To determine how individual hidden features react to quantization noise, the pipeline computes the encoder sensitivity floor. For each feature dictionary index i , the local sensitivity is defined by the L_2 norm of its corresponding input projection path in the encoder weight

matrix $\mathbf{W}_{\text{enc}} \in \mathbb{R}^{D_{\text{in}} \times d_{\text{sae}}}$. This calculation is handled entirely within a `float32` precision block to protect against infinity overflow errors:

$$\|\mathbf{J}_i\|_2 = \|\mathbf{W}_{\text{enc}}[:, i]\|_2 = \sqrt{\sum_{d=1}^{D_{\text{in}}} (\mathbf{W}_{\text{enc}}[d, i])^2} \quad (16)$$

This norm acts as a local Jacobian operator, tracking how violently an individual feature’s activation spikes when rounding errors (Δ) disrupt the dense input representations.

To verify the hypothesis of Sparsity Fragility, the framework measures the alignment between a feature’s structural sensitivity and its causal model importance. It builds a comprehensive topological correlation matrix by computing Spearman rank correlations (ρ) and p-values across all d_{sae} elements. The engine evaluates the base alignment between the Jacobian sensitivity norms and Fisher importance scores, and then dynamically evaluates these properties against active distortion arrays across all 8 quantization configurations (per-tensor and per-feature modes mapped across 8, 6, 4, and 2 bits). It evaluates correlations across multiple structural pathways:

$$\rho(\|\mathbf{J}\|_2, \mathbf{L}_2), \quad \rho(\mathbf{Attribution}, \mathbf{L}_2), \quad \rho(\|\mathbf{J}\|_2, \mathbf{D}_{\text{KL}}), \quad \rho(\mathbf{Attribution}, \mathbf{D}_{\text{KL}}) \quad (17)$$

Beyond global rank correlations, the script tracks localized top-tier feature alignment by evaluating a four-way intersection matrix among the top 50 neurons. This matrix computes the exact topological overlap percentages for:

- The top 50 causally vital features (Fisher) versus the top 50 structurally sensitive features (Jacobian).
- The top 50 most heavily distorted features in coordinate space (L_2) versus the top 50 causally vital features (Fisher).
- The top 50 features experiencing maximum distribution shift (D_{KL}) versus the top 50 causally vital features (Fisher).
- The top 50 maximum L_2 distorted features versus the top 50 structurally sensitive features (Jacobian).
- The top 50 maximum D_{KL} distorted features versus the top 50 structurally sensitive features (Jacobian).

6.3 Information-Theoretic Rank & Roy-Vetterli Spectral Entropy

To diagnose low-variance collapse without introducing threshold artifacts from independent channel noise, the framework replaces arbitrary hard cuts with an information-theoretic dimension analysis. The effective geometric rank of the representation space is calculated via Spectral Shannon Entropy, following the formulation established by Roy and Vetterli (2007).

Given a singular value spectrum $\sigma = \{\sigma_1, \sigma_2, \dots, \sigma_m\}$ extracted via SVD from either centered full-precision or quantized activation blocks, the pipeline isolates positive values ($\sigma > 0$) and computes an L_1 -normalized distribution profile where individual singular value amplitudes serve as density weights:

$$p_i = \frac{\sigma_i}{\sum_{j=1}^m \sigma_j} \quad (18)$$

Using this scale-invariant weight vector, the Shannon spectral entropy $H(\boldsymbol{\sigma})$ is calculated with a numerical floor of 10^{-12} to guarantee stability across dead dimensions:

$$H(\boldsymbol{\sigma}) = - \sum_{i=1}^m p_i \log(p_i + 10^{-12}) \quad (19)$$

The continuous effective rank of the matrix space is then derived by exponentiating the spectral entropy:

$$\text{Rank}_{\text{eff}}(\boldsymbol{\sigma}) = \exp(H(\boldsymbol{\sigma})) \quad (20)$$

Using this framework, the script calculates a scale-invariant collapse ratio across all 8 configurations. This ratio measures the structural collapse of low-variance components into zero under uniform grids:

$$\text{Collapse Ratio} = 1.0 - \frac{\text{Rank}_{\text{eff},q}}{\max(\text{Rank}_{\text{eff},fp}, 10^{-6})} \quad (21)$$

6.4 Angular Subspace Rotation & Geometric Deformity Statistics

To chart how rounding errors warp information routing, the framework tracks geometric orientation shifts. It extracts the principal angle cosine arrays stored during the cross-subspace SVD passes across rank bounds $k \in \{32, 64, 128\}$. To provide an intuitive geometric perspective, the cosine projections (σ_c) are transformed into explicit angular degrees:

$$\theta_c = \frac{180}{\pi} \cdot \arccos(\text{clip}(\sigma_c, -1.0, 1.0)) \quad (22)$$

The clipping boundary ensures that any floating-point precision anomalies are bounded within valid trigonometric inputs, preventing downstream NaN calculation breaks.

For each quantization configuration and subspace threshold k , the diagnostic engine calculates three statistical summary metrics:

1. **Mean Subspace Rotation (Mean(θ_c)):** Tracks the average angular displacement of the primary representation vectors.
2. **Maximum Subspace Drift (max(θ_c)):** Identifies the worst-case angular distortion experienced by any vector within the subspace boundary.
3. **Minimum Principal Alignment (min(σ_c)):** Pinpoints the lowest baseline cosine overlap between the full-precision and quantized subspace directions.

These structural deformity metrics are saved systematically inside a local `task4_part1_metrics.json` data repository.

6.5 Results and Interpretation

Placeholder section for Plots and Analyzes

7 Task 4, Part 2: Subspace-Preserving Quantization

7.1 Symmetric Signed Quantization Formulation

While uniform quantization in Task 2 relied on an unsigned configuration due to the non-negative properties of ReLU outputs ($z \geq 0$), the implementation of subspace-preserving quantization mandates a structural shift to a symmetric signed uniform quantization engine. The geometric decomposition framework centres the raw bottleneck representations around an empirical mean coordinate path ($\mathbf{z}_{\text{centered}} = \mathbf{z} - \boldsymbol{\mu}$), displacing the coordinate grid to surround the data’s central mass. This translation breaks the hard non-negativity constraint, introducing distinct negative validation attributes across latent paths.

To enforce quantization intervals symmetrically, for a designated target chunk bitwidth b , integer grid boundaries are assigned to the localized complement range $[q_{\min}, q_{\max}] = [-(2^{b-1}), 2^{b-1} - 1]$. The quantization scaling factor Δ determines the step grid width based on locked static calibration thresholds. Given a targeted maximum bound $V_{\max}$ and an operational protection ceiling floor of 10^{-8} to safeguard inactive paths against division anomalies, the transformations scale inputs according to:

$$\Delta = \max\left(\frac{V_{\max}}{2^{b-1} - 1}, 10^{-8}\right) \quad (23)$$

$$\hat{w} = \text{clip}\left(\left\lfloor \frac{w}{\Delta} \right\rfloor, -2^{b-1}, 2^{b-1} - 1\right), \quad \tilde{w} = \hat{w}\Delta \quad (24)$$

7.2 Geometric Subspace Decomposition and Routing Logic

The intervention framework splits centered representations along two geometric pathways using a target dictionary rank threshold vector $k \in \{32, 64, 128\}$. The full-precision hidden matrix $\mathbf{Z}$ is centered using the historical profile mean vector $\boldsymbol{\mu}$, and projected against the master full-precision dictionary singular vectors $\mathbf{U}_k \in \mathbb{R}^{m \times k}$ generated in Task 3. This maps load-bearing semantic directives into low-dimensional coordinate spaces:

$$\mathbf{Z}_{\text{centered}} = \mathbf{Z} - \boldsymbol{\mu} \quad (25)$$

$$\mathbf{P} = \mathbf{Z}_{\text{centered}} \mathbf{U}_k \quad (26)$$

The high-variance geometric core is subsequently reconstructed inside the bottleneck domain, enabling the script to isolate the low-variance residual noise matrix $\mathbf{Z}_{\text{res}}$:

$$\mathbf{Z}_{\text{imp}} = \mathbf{P} \mathbf{U}_k^\top \quad (27)$$

$$\mathbf{Z}_{\text{res}} = \mathbf{Z}_{\text{centered}} - \mathbf{Z}_{\text{imp}} \quad (28)$$

Once isolated, the information channels are routed through a mixed-precision execution scheme. The core structural coordinates vector $\mathbf{P}$ is protected using high-precision uniform 8-bit quantization to prevent structural degradation and low-variance collapse. Conversely, the non-aligned residual vectors in $\mathbf{Z}_{\text{res}}$ are compressed aggressively using 2-bit or 4-bit quantization arrays to minimize memory consumption. Following de-quantization, the components are combined and the global center $\boldsymbol{\mu}$ is added back to restore the baseline scale:

$$\hat{\mathbf{Z}}_{\text{final}} = \tilde{\mathbf{Z}}_{\text{imp_8bit}} + \tilde{\mathbf{Z}}_{\text{res_lowbit}} + \boldsymbol{\mu} \quad (29)$$

7.3 Static Training Calibration and Memory Footprint Management

To secure a robust, unbiased evaluation across held-out corpora, the framework locks in its calibration parameters before performing experimental execution passes. The pipeline reuses the exact 204.8k token calibration stream (`CALIBRATION_TOKENS = 4096 * 50`) assigned during Task 2. This calibration script maps the unquantized baseline matrix and extracts the reference bounds over the target distribution. The training array is centered and projected against the master basis vectors to calculate the absolute static scale maximums:

$$\text{max_val_imp} = \max(|\mathbf{Z}_{\text{calib_centered}} \mathbf{U}_k|) \quad (30)$$

$$\text{max_val_res} = \max(|\mathbf{Z}_{\text{calib_centered}} - \mathbf{Z}_{\text{imp_calib}}|) \quad (31)$$

To implement this calibration framework within an 8GB VRAM compute boundary without causing out-of-memory faults, the pipeline uses a host-managed memory hierarchy. Rather than materializing the dense token representations directly in GPU hardware, the pipeline streams data blocks in small chunks, converts them to sparse states, and streams the outputs to host RAM into a centralized collection list (`Z_calib_list`). This offloading strategy protects the limited GPU VRAM footprint on our local NVIDIA RTX 4060 hardware.

7.4 Intervention Infrastructure and Multidimensional Metrics

Model intervention relies on the runtime hook architecture provided by `TransformerLens`. The custom patching hook function (`make_subspace_quant_hook`) intercepts the forward pass at the zero-indexed Layer 2 residual hook location (`blocks.2.hook_resid_post`). The hook flattens the input hidden tensors, executes the dictionary encoding pass, applies the mixed-precision subspace quantization logic, and replaces the baseline representations before the model resumes the remaining layer executions.

Downstream performance indicators are logged to a dedicated `metrics_m[size]_task4.json` repository across 12 distinct experimental variations (combining 2 bottleneck capacities, 3 subspace rank limits, and 2 low-bit configurations). The pipeline monitors global downstream language modeling degradation via Perplexity evaluated across the 2M token stream (`ppl_tokens_2M.pt`). Simultaneously, local geometric changes are tracked on the 4096 held-out token batch (`sds_tokens_4096.pt`) by logging Mean Squared Error (MSE), Linear Centered Kernel Alignment (CKA), and a multi-threshold profile of Subspace Distortion Scores (SDS) evaluated for all standard testing validation limits ($k \in \{32, 64, 128\}$).

7.5 Results and Interpretation

Placeholder section for Plots and Analyzes

8 Discussion & Unified Insights

Placeholder Section: Come back to expand on unified insights and synthesis of tasks.

9 Conclusion & Future Work

Placeholder Section: Come back to expand on conclusions and prospective bonus tasks.

10 References

- *Placeholder: Add citations for Neel Nanda, Anthropic’s Scaling Monosemanticity, TransformerLens, SAELens, etc.*